SHALLOW COPY AND DEEP COPY IN C++

- In general, creating a copy of an object means to create an exact replica of the object having the same literal value, data type, and resources.
    - Copy Constructor
    - Default assignment operator

```
// Copy Constructor
T a(b);
or
T a = b;

// Default assignment operator
T a;
a = b;
```

- Depending upon the resources like dynamic memory held by the object, either we need to perform **Shallow Copy** or **Deep Copy** in order to create a replica of the object. In general, if the variables of an object have been dynamically allocated, then it is required to do a Deep Copy in order to create a copy of the object.

SHALLOW COPY

- In shallow copy, an object is created by simply copying the data of all variables of the original object. This works well if none of the variables of the object are defined in the heap section of memory.

- If some variables are dynamically allocated memory from the heap section, then the copied object variable will also reference the same memory location. This will create ambiguity and run-time errors, dangling pointer. Since both objects will reference to the same memory location, then change made by one will reflect those change in another object as well.

- Since we wanted to create a replica of the object, this purpose will not be filled by Shallow copy.

```
#include <iostream>
using namespace std;
class box {
    int length;
    int breadth;
    int height;
public:
    void set_dimensions(int length1, int
breadth1, int height1)
    {
        length = length1;
        breadth = breadth1;
        height = height1;
    }
```

```
void show_data()
    {
    cout << " Length = " << length
    << "\n Breadth = " << breadth
    << "\n Height = " << height<< endl;
    }
};

int main()
{
    box B1, B3;
    B1.set_dimensions(14, 12, 16);
    B1.show_data();
    box B2 = B1;
    B2.show_data();
    B3 = B1;
    B3.show_data();

    return 0;
}
```

Shallow Copy

```
#include <iostream>
#include <cstring>
using namespace std;

class Student {
    char *name;   // pointer -> stored in heap
public:
    Student(const char *n) {
        name = new char[strlen(n) + 1];  // allocate
memory in heap
        strcpy(name, n);
    }

// Shallow Copy Constructor
    Student(const Student &s) {
        name = s.name;  // only pointer copied, not memory
    }

 void setName(const char *n) {
        strcpy(name, n);  // modifies the same heap memory
    }
```

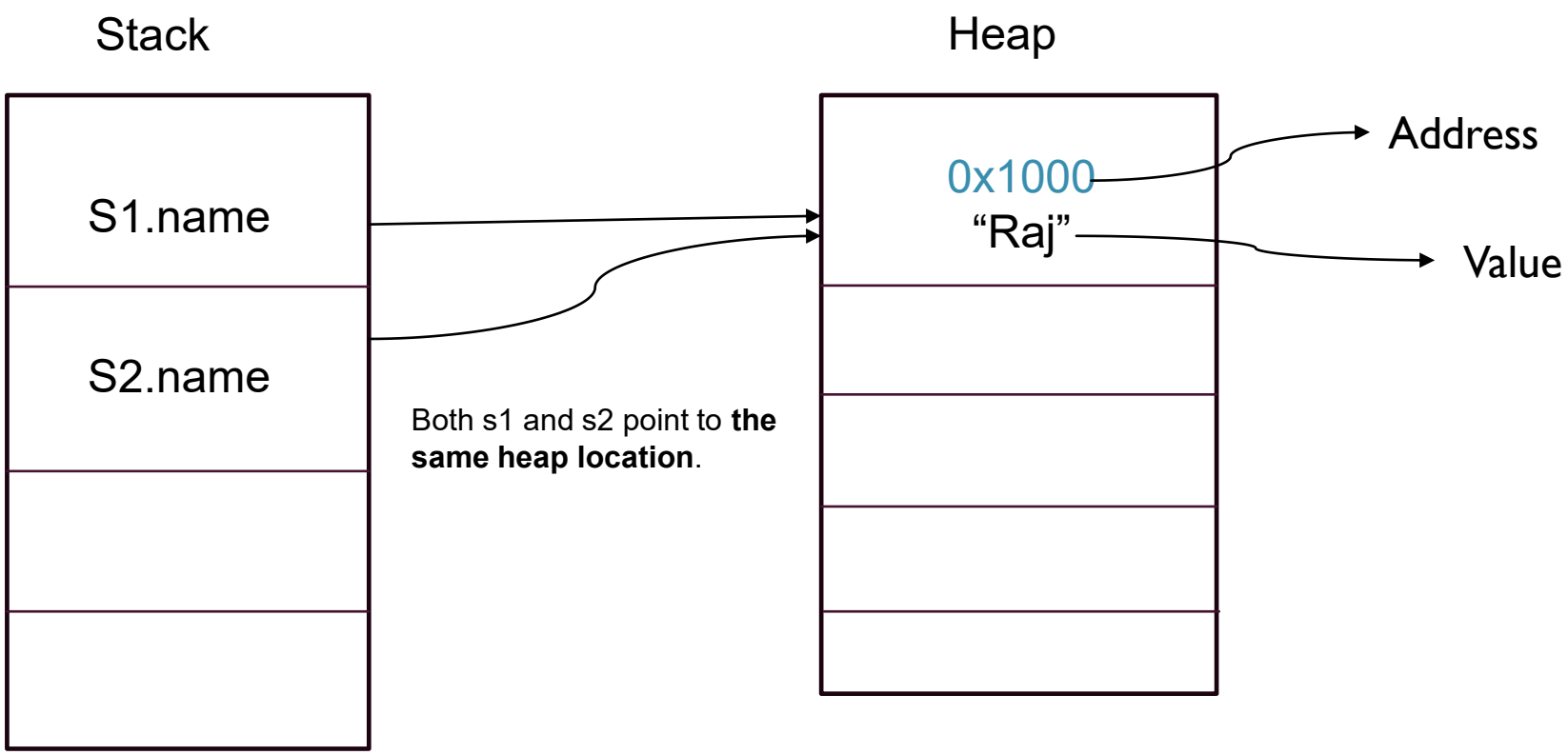
```
    void print() {
        cout << "Name: " << name << endl;
    }

    ~Student() {
        delete[] name; // frees heap memory
    }
};

int main() {
    Student s1(“Raj");   // allocates heap memory for “Raj"

    Student s2 = s1;       // shallow copy (pointer copied)

    cout << "Before change:" << endl;
    s1.print();   // Raj
    s2.print();   // Raj
    s1.setName("Patel");   // modifies heap memory
    cout << "After change:" << endl;
    s1.print();   // Patel
    s2.print();   // Patel (unexpected!)
    return 0;
}
```



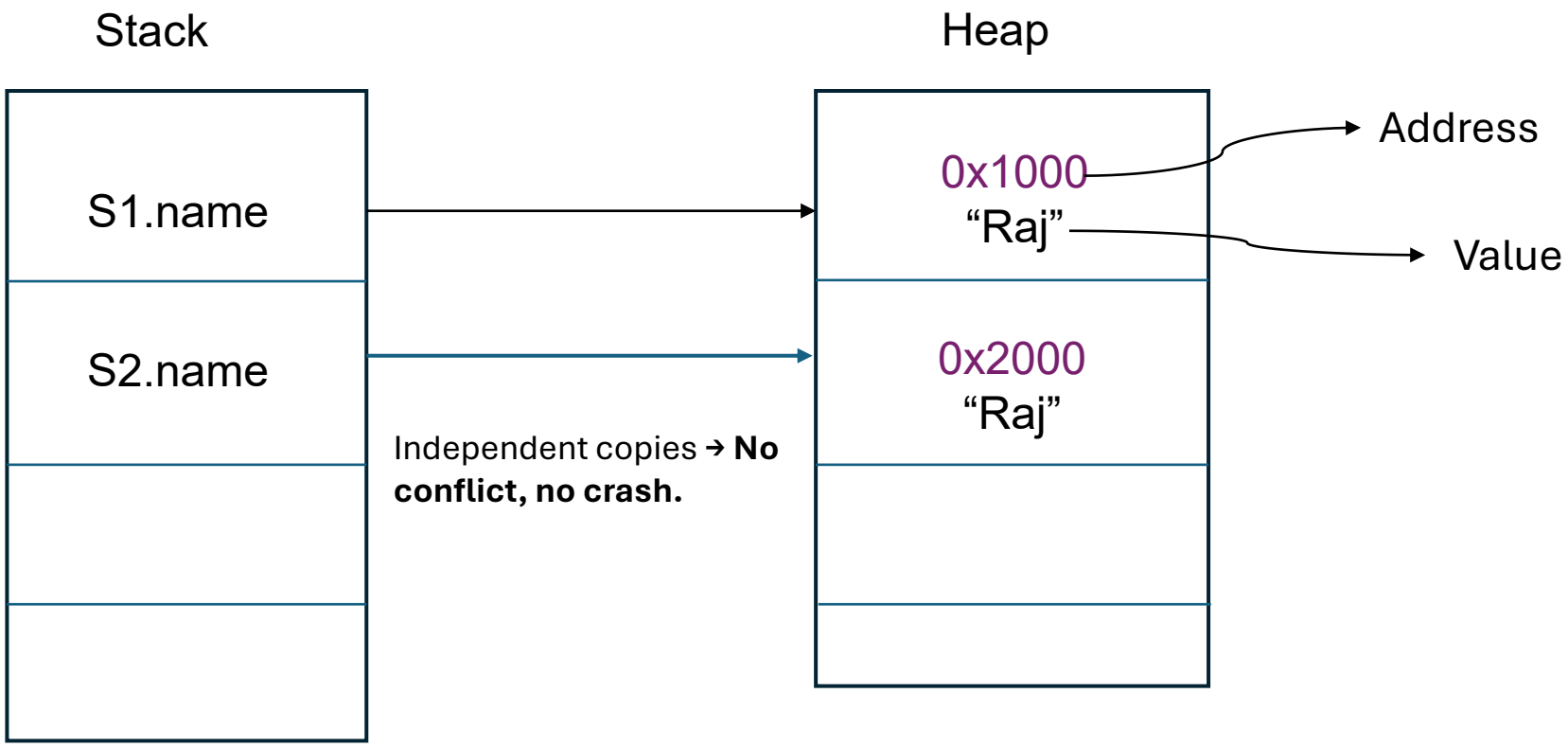
Problem:

- Changes in one reflect in the other.
- When destructors run → both try to `delete` the same → delete[] runs **twice** on the same memory address (0x1000) → ❌ Dangling pointer / crash memory → runtime error

DEEP COPY

- In Deep copy, an object is created by copying data of all variables, and it also allocates similar memory resources with the same value to the object. In order to perform Deep copy, we **need to explicitly define the copy constructor** and **assign dynamic memory** as well, if required. Also, it is required to dynamically allocate memory to the variables in the other constructors, as well.

```
// Deep Copy Constructor
Student(const Student &s) {
    name = new char[strlen(s.name) + 1]; // new memory
    strcpy(name, s.name);                // copy content
}
```



So, in short:

- **Shallow copy** = copies only pointer (both objects share same heap data → risk).
- **Deep copy** = allocates new memory and copies actual content (safe & independent).

SHALLOW COPY VS DEEP COPY

	Shallow Copy	Deep Copy
1.	When we create a copy of object by copying data of all member variables as it is, then it is called shallow copy	When we create an object by copying data of another object along with the values of memory resources that reside outside the object, then it is called a deep copy
2.	A shallow copy of an object copies all of the member field values.	Deep copy is performed by implementing our own copy constructor.
3.	In shallow copy, the two objects are not independent	It copies all fields, and makes copies of dynamically allocated memory pointed to by the fields
4.	It also creates a copy of the dynamically allocated objects, not the actual objects themselves.	If we do not create the deep copy in a rightful way then the copy will point to the original, with disastrous consequences.